

《并行计算》期末综合模拟试卷

(按最新考试通知命制: 闭卷 · 问答题 · 第2-6章各2题 · 只考重点)

考试时间: 120 分钟

题数: 10 题

每题 10 分, 满分 100 分

建议每题 ≤ 12 分钟

答题要求: ① 闭卷作答, 不得查阅任何资料; ② **算法执行过程题**必须写出每一步的中间结果; ③ **程序改错题**需先指出错误并说明原因, 再写出修改后的语句; ④ 计算题需给出推导过程, 只写结果不给满分。参考答案附于试卷末页, 自测时请先独立完成全卷再核对。

一、(10 分)

第2章

概念解释

- (a) 请详述均匀存储访问 (UMA) 与非均匀存储访问 (NUMA) 多处理器系统各自的特点, 并说明二者的本质区别。(4 分)
- (b) 解释互连网络指标"等分宽度 (对剖宽度)"的含义。(2 分)
- (c) 分别给出 p 个节点的二维正方形网格 (无环绕链路) 和 d 维超立方 ($p = 2^d$) 的等分宽度, 并简要说明理由。(4 分)

二、(10 分)

第2章

计算

设某程序顺序执行时间 $T_1 = N^2$, 在 p 个处理器上的并行执行时间 $T_p = N^2/p + 2N$, 其中 N 为问题规模。

- (1) 求固定负载为 N 时的加速比, 并讨论 $p \rightarrow \infty$ 时的结果; (4 分)
- (2) 求固定时间为 T 时的加速比, 并讨论其随 p 的变化趋势; (4 分)
- (3) 当 $N = 100$ 、 $p = 10$ 时, 计算固定负载下的加速比与效率。(2 分)

三、(10 分)

第3章

算法执行过程

在 3×3 处理器阵列上用 Cannon 算法计算 $C = A \cdot B$, 初始时处理器 P_{ij} 持有元素 A_{ij} 与 B_{ij} (下标从 0 开始), 其中:

$$\begin{vmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{vmatrix} = A, \quad \begin{vmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \end{vmatrix} = B$$

- (a) 写出 Cannon 算法的初始对准规则, 并给出对准完成后 $P_{1,2}$ 所持有的 A 、 B 元素; (3 分)
- (b) 逐轮写出 $P_{1,2}$ 的本地乘加运算, 以及每轮移位时它把 A 、 B 元素发送给谁、从谁接收; (5 分)
- (c) 给出最终 $C_{1,2}$ 的值, 并验证其等于 $\sum_k A_{1,k}B_{k,2}$ 。(2 分)

四、(10 分)

第3章

程序改错

以下 MPI 程序拟在 2 个进程间互换问候消息 (假设 $\text{comm_sz} = 2$) :

```
#include <stdio.h>
#include <string.h>
#include <mpi.h>

int main(void) {
    char send_msg[100], recv_msg[100];
    int my_rank, comm_sz, partner;

    MPI_Init(NULL, NULL);
```

```

MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);      /* comm_sz == 2 */
partner = 1 - my_rank;
sprintf(send_msg, "Greetings from process %d!", my_rank);

MPI_Recv(recv_msg, 100, MPI_CHAR, partner, 0,
         MPI_COMM_WORLD, MPI_STATUS_IGNORE);
MPI_Send(send_msg, strlen(send_msg), MPI_CHAR, partner, 0,
         MPI_COMM_WORLD);

printf("Process %d received: %s\n", my_rank, recv_msg);
MPI_Finalize();
return 0;
}

```

- (a) 该程序存在两处错误（一处导致程序无法继续运行，一处导致输出可能不正确），请分别指出并说明原因；（5分）
- (b) 给出修改方案。其中对"无法继续运行"的错误，请给出两种不同的修复方法。（5分）

五、(10 分) 第4章 程序优化

用 4 个 Pthreads 线程计算 8 行 $\times$ 8 000 000 列的矩阵 - 向量乘法 $y = A \cdot x$ （double 占 8 字节，缓存行 64 字节，各线程运行在不同核上）：

```

double A[8][8000000], x[8000000], y[8];
int thread_count = 4;

void *Pth_mat_vect(void *rank) {
    long my_rank = (long) rank;
    int local_m = 8 / thread_count;          /* = 2 */
    int first = my_rank * local_m;
    int last = first + local_m - 1;

    for (int i = first; i <= last; i++) {
        y[i] = 0.0;
        for (int j = 0; j < 8000000; j++)
            y[i] += A[i][j] * x[j];          /* 反复写 y[i] */
    }
    return NULL;
}

```

- (a) 该程序结果正确，但多线程加速效果很差。指出性能问题的名称并解释其成因；（4分）
- (b) 给出两种优化方案并写出关键代码；（4分）
- (c) 说明两种方案为何有效。（2分）

六、(10 分)

第4章

概念解释

(a) 两个线程按下表顺序获取两把互斥锁：

时刻	线程 0	线程 1
t_0	lock(&mut0)	lock(&mut1)
t_1	lock(&mut1)	lock(&mut0)

会发生什么？解释其成因，并给出一种预防方法。（5 分）

(b) 简述如何仅用互斥锁 + 条件变量实现一个可复用的路障（barrier）：说明所需共享变量、线程到达路障时的动作、最后一个到达线程的动作。（5 分）

七、(10 分)

第5章

算法执行过程

给定序列 (15, 1, 9, 4, 12, 7, 2, 17, 10, 5, 18, 3, 13, 8, 16, 6, 11, 14) 和 3 个处理器 P1、P2、P3（依次分得前、中、后各 6 个元素），运行 PSRS（正则采样排序）算法。请按“均匀划分 → 局部排序 → 正则采样 → 样本排序 → 选主元 → 主元划分 → 全局交换与多路归并”七个步骤写出每一步的结果，并明确 P1 最终负责归并得到的数据段。

八、(10 分)

第5章

程序改错

以下 OpenMP 程序拟用莱布尼茨级数 $\pi = 4 \cdot (1 - 1/3 + 1/5 - 1/7 + \dots)$ 并行计算 π ：

```
double sum = 0.0, factor;
long n = 100000000;

# pragma omp parallel for num_threads(4)
for (long i = 0; i < n; i++) {
    if (i % 2 == 0) factor = 1.0;
    else factor = -1.0;
    sum += factor / (2*i + 1);
}
double pi = 4.0 * sum;
```

(a) 该程序每次运行结果都不同且明显错误。指出其中两处并行性错误，并说明各自导致的后果；（4 分）

(b) 只修改编译指导语句，写出改正后的完整 #pragma 行；（3 分）

(c) 改正后，并行结果与串行结果在最后几位仍可能不同。这是错误吗？为什么？（3 分）

九、(10 分)

第6章

概念解释 + 程序改错

(a) 说明 CUDA 执行模型中 grid、block、thread、warp、SM 五个概念的含义，以及它们之间的组织与映射关系。（4 分）

(b) 以下向量加法程序含有三处错误，请指出并改正（ $n = 1\,000\,000$ ）：（6 分）

```
__global__ void Vec_add(const float x[], const float y[],
                        float z[], const int n) {
    int i = threadIdx.x;
    z[i] = x[i] + y[i];
}

int main(void) {
    ... /* 已完成 cudaMalloc 和输入数据 Host→Device 复制 */
    int n = 1000000;
    int th_per_blk = 256;
    int blk_ct = (n + th_per_blk - 1) / th_per_blk; /* = 3907 */
}
```

```
Vec_add<<<blk_ct, th_per_blk>>>(d_x, d_y, d_z, n);

cudaMemcpy(h_z, d_z, n * sizeof(float),
           cudaMemcpyHostToDevice);
...
}
```

十、(10 分)

第6章

算法执行过程

画出基于 Batcher 比较器的 8 输入双调排序网络，输入为 (5, 7, 1, 6, 3, 8, 2, 4)。要求：

- (a) 按级（阶段）画出网络结构，标出每个比较器的方向（升序/降序）及其输入、输出数据，即给出每一级结束后的完整序列；（8 分）
- (b) 说明各级比较方向的排布规律（何时构造双调序列、何时进行双调归并）。（2 分）

一、UMA / NUMA 与等分宽度

(a) UMA: 所有处理器通过同一互连（总线/交叉开关）访问共享主存，访问任一存储单元延迟相同（对称多处理器 SMP）；结构与编程简单，但互连带宽随处理器数增加成为瓶颈，可扩展性差 (2分)。NUMA: 存储器物理分布于各处理器节点，访问本地存储快、远程存储慢，访问时间取决于数据位置；可扩展性好，但性能依赖数据局部性 (2分)。本质区别：访存延迟是否均匀一致。

(b) 等分宽度：把网络节点分成相等两半时，所需切断的最少链路数；衡量网络的整体通信带宽与瓶颈 (2分)。

(c) 二维正方形网格：沿中线竖切恰断 $\sqrt{p}$ 条链路，等分宽度 = $\sqrt{p}$ (2分)；d 维超立方：由两个 d-1 维子立方对应节点相连构成，两半之间恰有 $p/2$ 条对应边，故等分宽度 = $p/2$ (2分)。

对应题库：EXAM-01、HW-CH2-2.13、HW-CH2-2.14

二、加速比（固定负载 / 固定时间）

(1) $S = T_1/T_p = N^2/(N^2/p + 2N) = Np/(N + 2p)$ 。 $p \rightarrow \infty$ 时 $S \rightarrow N/2$ ：固定负载下，开销项 $2N$ 不随 p 减小，加速比存在上限 $N/2$ (Amdahl 型) (4分)。

(2) 令 $N^2/p + 2N = T$ ，解得 $N^2 + 2pN - pT = 0 \Rightarrow N = -p + \sqrt{(p^2 + pT)} \approx \sqrt{(pT)}$ (当 $T \gg p$)。此时串行时间 $T_1 = N^2 \approx pT$ ，故固定时间加速比 $S = T_1/T \approx p$ ：随处理器数近似线性增长 (Gustafson 型，扩大问题规模可保持高效率，弱可扩展) (4分)。

(3) $S = 100 \times 10 / (100 + 20) = 1000 / 120 \approx 8.33$ ； $E = S/p \approx 0.83$ (2分)。

对应题库：EXAM-04、REF3-6（同型换数）

三、Cannon 算法 ($P_{1,2}$)

(a) 初始对准：A 的第 i 行循环左移 i 步；B 的第 j 列循环上移 j 步 (2分)。对准后 P_{ij} 持 $A_{i,(i+j) \bmod 3}$ 、 $B_{(i+j) \bmod 3, j}$ ，故 $P_{1,2}$ 持 $A_{1,0} = 4$ ， $B_{0,2} = 2$ (1分)。

(b) 每轮"先本地乘加，再 A 左移一格、B 上移一格"（均带环绕）（每轮至少写出运算与收发对象，共5分）：

- 第 1 轮： $c += 4 \times 2 = 8$ ；随后 A 元素发送给左邻 $P_{1,1}$ 、从右邻 $P_{1,0}$ （环绕）接收；B 元素发送给上邻 $P_{0,2}$ 、从下邻 $P_{2,2}$ 接收。
- 第 2 轮：此时持 $A_{1,1} = 5$ 、 $B_{1,2} = 1$ ， $c += 5 \times 1 \Rightarrow c = 13$ ；再同样左移/上移。
- 第 3 轮：持 $A_{1,2} = 6$ 、 $B_{2,2} = 0$ ， $c += 6 \times 0 \Rightarrow c = 13$ 。

(c) $C_{1,2} = 13$ ；验证： $\sum_k A_{1,k} B_{k,2} = 4 \times 2 + 5 \times 1 + 6 \times 0 = 13$ ✓ (2分)。

对应题库：EXAM-07、HW2-3.A、LAB-2

四、MPI 程序改错

(a) 错误 1：两个进程都"先 MPI_Recv 后 MPI_Send"。MPI_Recv 是阻塞调用，双方都在等待对方发送而无人发送，程序死锁（悬挂） (3分)。错误 2：发送长度用 `strlen(send_msg)`，漏发字符串结尾 '\0'，接收缓冲区中的字符串可能不以 0 结束，printf 输出可能出现乱码/越界读 (2分)。

(b) 死锁修复（写出任两种）（每种2分）：

- 按 rank 交错收发：`if (my_rank == 0) { Send; Recv; } else { Recv; Send; }`，保证一方先发送；
- 改用 `MPI_Sendrecv(send_msg, strlen(send_msg)+1, MPI_CHAR, partner, 0, recv_msg, 100, MPI_CHAR, partner, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE)`，由 MPI 内部安全调度；
- （亦可）用非阻塞 `MPI_Irecv/MPI_Isend + MPI_Wait`。

长度修复：发送长度改为 `strlen(send_msg) + 1`（或以 MAX 长度发送，'\0' 随数据到达）（1分）。

对应题库：MOCK1-SA-5、HW1-3.1、MOCK2-CH-4

五、伪共享优化

(a) 问题：伪共享（false sharing）。y 共 8 个 double = 64 字节，恰好落在同一（或两个）缓存行内；4 个线程虽写 y 的不同元素，但这些元素处于同一缓存行，缓存一致性协议使各核的该缓存行反复互相无效化，每次 `y[i] +=` 都可能触发一致性通信，性能急剧下降（4分）。

(b) 方案一（线程私有累加）：内层用局部变量累加，循环结束只写一次 `y[i]`（2分）：

```
double s = 0.0;
for (int j = 0; j < 8000000; j++) s += A[i][j] * x[j];
y[i] = s;    /* 每行仅写共享数组一次 */
```

方案二（缓存行填充 padding）：把 y 扩为 `double y[8][8]`，线程使用 `y[i][0]`，使每个分量独占一个 64 字节缓存行（2分）。

(c) 两者都使不同线程不再频繁写同一缓存行：私有累加把对共享行的写从 8×10^6 次降到 2 次；padding 让各线程的写落在不同缓存行，消除一致性乒乓。二者性能相近，均显著快于原程序（2分）。

对应题库：HW-CH4-4.17、HW-CH4-4.18、HW-CH4-4.19

六、死锁与条件变量路障

(a) 发生死锁： t_0 后线程 0 持有 `mut0`、线程 1 持有 `mut1`； t_1 时线程 0 等 `mut1`、线程 1 等 `mut0`，形成循环等待，双方永远阻塞（3分）。预防：让所有线程按相同的全局顺序获取多把锁（如都先 `mut0` 后 `mut1`）；或用 `trylock`+回退（2分）。

(b) 共享变量：计数器 `counter`、互斥锁 `mutex`、条件变量 `cond`（可加轮次号防虚假唤醒/复用错乱）（1分）。到达线程：`lock(mutex)`；`counter++`；若 `counter < 线程数`，则在 `while` 循环中 `pthread_cond_wait(&cond, &mutex)`（wait 自动释放锁、被唤醒后重新持锁再检查条件）（2分）。最后一个到达的线程：把 `counter` 清零（进入下一轮次），并 `pthread_cond_broadcast(&cond)` 唤醒所有等待线程；随后各线程解锁离开，路障可重复使用（2分）。

对应题库：HW-CH4-4.9、LAB-4

七、PSRS 执行过程

- 均匀划分：P1: 15,1,9,4,12,7 | P2: 2,17,10,5,18,3 | P3: 13,8,16,6,11,14（1分）
- 局部排序：P1: 1,4,7,9,12,15 | P2: 2,3,5,10,17,18 | P3: 6,8,11,13,14,16（2分）
- 正则采样（每段取下标 0,2,4，间隔 $n/p^2 = 2$ ）：P1: 1,7,12 | P2: 2,5,17 | P3: 6,11,14（2分）
- 样本排序（9 个）：1,2,5,6,7,11,12,14,17（1分）
- 选主元（取第 p、2p 个，即排序后下标 3、6 的样本）：6 和 12（1分）
- 主元划分：P1: {1,4}|{7,9,12}|{15}；P2: {2,3,5}|{10}|{17,18}；P3: {6}|{8,11}|{13,14,16}（2分）
- 全局交换与多路归并：P1 收集所有" ≤ 6 "段 {1,4},{2,3,5},{6} → 归并得 1,2,3,4,5,6；P2 得 7,8,9,10,11,12；P3 得 13,14,15,16,17,18（1分）

P1 最终负责数据段：1,2,3,4,5,6（三段桶恰好均衡，各 6 个）。

对应题库：EXAM-05、REF5-3、LAB-6

八、OpenMP π 程序改错

(a) 错误 1：sum 为共享变量，多线程同时执行 `sum += ...` 产生数据竞争（读-改-写交错丢失更新），结果错误且每次不同（2分）。错误 2：factor 在并行区外声明、默认共享，一个线程刚写入的 factor 可能被另一线程改写后才

被读取，符号错乱 (2分)。

(b) `#pragma omp parallel for num_threads(4) reduction(+:sum) private(factor)` (3分: `reduction` 2分、`private` 1分) (`factor` 每次迭代内按 `i` 重新赋值，故 `private` 即可)。

(c) 不是错误。浮点加法不满足严格结合律，`reduction` 把求和分成各线程私有部分和再合并，改变了加法分组/顺序，舍入误差累积路径不同，故与串行结果可在最后几位不同；两者都是对真值的合法近似 (3分)。

对应题库: MOCK1-PROG-1、HW-CH5-5.5、MOCK1-FILL-10

九、CUDA 概念与改错

(a) 一次 `kernel` 启动形成一个 `grid`；`grid` 由若干 `block` (线程块) 组成；`block` 由若干 `thread` 组成。硬件将 `block` 调度到 `SM` (流多处理器) 上执行——一个 `SM` 可驻留多个 `block`，但一个 `block` 不会拆到多个 `SM`；`block` 内线程按每 32 个一组划分为 `warp`，`warp` 是 `SM` 的调度单位，以 `SIMT` 方式执行 (4分)。

(b) 三处错误 (每处指出+改正 2分)：

- 索引公式错: `int i = threadIdx.x`；只是块内索引，各 `block` 会重复计算前 256 个元素。应为 `int i = blockIdx.x * blockDim.x + threadIdx.x`；
- 缺少边界判断: 总线程数 $3907 \times 256 = 1\,000\,192 > n$ ，多余线程会越界访存。应改为 `if (i < n) z[i] = x[i] + y[i]`；
- `cudaMemcpy` 方向错: 取回结果应为 `cudaMemcpyDeviceToHost` (写成 `HostToDevice` 会把主机未初始化内容拷去设备，`h_z` 得不到结果)。

对应题库: PPT6-1、PPT6-2、PPT6-3、LAB-7

十、双调排序网络 (输入 5,7,1,6,3,8,2,4)

比较方向规律: 分 3 个大阶段 ($k = 2, 4, 8$)，第 k 阶段内按 $j = k/2, \dots, 2, 1$ 逐级比较距离 j 的元素对；元素 i 所在比较器方向由 $(i \text{ AND } k)$ 决定——为 0 取升序、否则取降序。前两个阶段把序列逐步构造成双调序列，最后阶段 ($k=8$) 对整体双调序列做双调归并 (2分)。各级输出 (每级1分，最终有序+2分)：

- $k=2, j=1$ ($\uparrow \downarrow \uparrow \downarrow$) : 5,7, 6,1, 3,8, 4,2
- $k=4, j=2$: 5,1,6,7, 4,8,3,2
- $k=4, j=1$: 1,5,6,7 (升), 8,4,3,2 (降) —— 整体已成双调序列
- $k=8, j=4$: 1,4,3,2, 8,5,6,7
- $k=8, j=2$: 1,2,3,4, 6,5,8,7
- $k=8, j=1$: 1,2,3,4,5,6,7,8 (有序 ✓)

画图时: 8 条水平数据线，按上述 6 级放置比较器 (j 为比较距离)，每个比较器标出两输入与升/降序输出。

对应题库: EXAM-06、REF5-1

命题说明: 本卷严格按最新通知命制——第 2-6 章各 2 题，题型覆盖概念解释、程序改错、程序优化、算法执行过程四类；全部考点取自题库 S/A 级重点 (每题末尾已标注对应题库题号，可回题库追练同型题)。算法过程题 (三、七、十) 与计算题 (二) 的数据为全新命制，答案均经逐步验算。